

Searching Algorithms

Learning Objectives

- Implement linear search and binary search algorithms
- Analyze the performance advantage of binary search on sorted data
- Determine which search algorithm to use based on data structure and requirements

Engage (5-7 minutes)

Play a guessing game: one student thinks of a number between 1 and 100. The class guesses. First strategy: random guesses (simulating linear search). Second strategy: always guess the midpoint and eliminate half (simulating binary search). Record how many guesses each strategy takes—binary search consistently wins.

Explore (10-12 minutes)

Students work with a sorted list of 1000 student records. They perform linear search (checking each record) and binary search (halving the search space) to find specific entries. They record comparisons for each method across different target positions (first, middle, last, not found) and calculate the maximum comparisons for each algorithm.

Explain (10-12 minutes)

Linear search examines each element sequentially until the target is found or the list is exhausted— $O(n)$ time complexity. Binary search repeatedly divides a sorted array in half, comparing the target to the middle element— $O(\log n)$ time complexity. Binary search's dramatic speed advantage comes with a prerequisite: the data must be sorted. For unsorted data, linear search is the only option without preprocessing.

Elaborate (8-10 minutes)

Analyze real-world search applications: database indexing (B-trees), web search engines (inverted indexes), and file system lookups. Discuss hybrid approaches—interpolation search for uniformly distributed data, jump search as a middle ground. Examine how sorted data enables efficiency: the cost of preprocessing (sorting once) is amortized over many searches.

Evaluate (5-8 minutes)

Students implement both search algorithms and test them on datasets of varying sizes. Performance comparison: they measure and graph time taken for each algorithm as input size increases. Written analysis: justify which algorithm to use when the data is unsorted but will be searched repeatedly.